



**BURSA TECHNICAL
UNIVERSITY**

Mühendislik ve Doğa Bilimleri Fakültesi

Bilgisayar Mühendisliği

Paralel Programlama

Paralel Point-in-Polygon Tespiti

Akif Adnan

20360859106

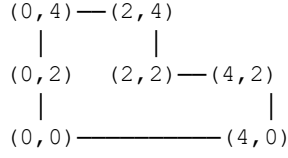
12 Haziran 2026

1. Problem Tanımı

Point-in-Polygon (PIP) problemi şu soruyu yanıtlar: verilen bir (px, py) noktası, köşeleri bilinen bir poligonun içinde mi, yoksa dışında mı? Bu proje, hem konveks hem konkav poligonlar için geçerli olan Ray Casting (Işın İzleme) algoritmasını ele almakta ve Java ExecutorService ile thread tabanlı paralelleştirme uygulamaktadır.

Seçilen Test Poligonu — Konkav L-Şekli

Konkav (içbükey) L-şekli seçilmiştir. Bu poligon, basit merkez-uzaklık veya sınırlayıcı kutu testlerinin yanlış sonuç vereceği bir yapıdır; dolayısıyla gerçek geometrik algoritma zorunludur.



Köşeler: $\{(0,0), (4,0), (4,2), (2,2), (2,4), (0,4)\}$

2. Kullanılan Fonksiyonlar ve Çalışma Prensibi

2.1 Ray Casting Algoritması

Test noktasından (px, py) $+\infty$ yönünde yatay bir ışın gönderilir. Bu ışının poligon kenarlarını kaç kez kestiği sayılır:

- Tek sayıda kesişim $\rightarrow$ nokta İÇERİDE
- Çift sayıda kesişim $\rightarrow$ nokta DIŞARIDA

Algoritmanın zaman karmaşıklığı $O(n)$ olup n poligon köşe sayısıdır.

```
public boolean isInside(double px, double py) {
    boolean inside = false;
    int j = n - 1;
    for (int i = 0; i < n; i++) {
        boolean yCondition = (polyY[i] > py) != (polyY[j] > py);
        if (yCondition) {
            double xIntersect = (polyX[j] - polyX[i]) * (py - polyY[i])
                / (polyY[j] - polyY[i] + polyX[i]);
            if (px < xIntersect) inside = !inside;
        }
        j = i;
    }
    return inside;
}
```

2.2 Sıralı (Sequential) Kontrol

sequentialCheck() fonksiyonu, verilen tüm test noktalarını tek bir döngüde sırayla işler. Kıyaslama (baseline) ölçümü için kullanılır.

```

public boolean[] sequentialCheck(double[] testX, double[] testY) {
    boolean[] results = new boolean[testX.length];
    for (int i = 0; i < testX.length; i++)
        results[i] = isInside(testX[i], testY[i]);
    return results;
}

```

2.3 Paralel Kontrol — ExecutorService ile Thread Havuzu

parallelCheck() fonksiyonu M adet test noktasını T thread'e eşit parçalara böler. Her thread kendi parçasını bağımsız olarak işler. Thread'ler arasında veri paylaşımı veya senkronizasyon gerektiren ortak bir durum (shared mutable state) yoktur; bu da senkronizasyon maliyetini sıfıra indirir.

```

public boolean[] parallelCheck(double[] testX, double[] testY, int numThreads)
    throws InterruptedException, ExecutionException {
    int m = testX.length;
    boolean[] results = new boolean[m];
    ExecutorService executor = Executors.newFixedThreadPool(numThreads);
    List<Future<?>> futures = new ArrayList<>();

    int chunkSize = (m + numThreads - 1) / numThreads;
    for (int t = 0; t < numThreads; t++) {
        final int start = t * chunkSize;
        final int end = Math.min(start + chunkSize, m);
        if (start >= m) break;
        futures.add(executor.submit(() -> {
            for (int i = start; i < end; i++)
                results[i] = isInside(testX[i], testY[i]);
        }));
    }
    for (Future<?> f : futures) f.get();
    executor.shutdown();
    return results;
}

```

3. Doğruluk Testleri ve Çalışma Örnekleri

Aşağıdaki noktalar L-şekli konkav poligon üzerinde test edilmiş; tüm sonuçlar beklentiyle tam olarak örtüşmüştür.

Nokta (px, py)	Beklenen	Sonuç
(1.0, 1.0)	İÇERİDE	İÇERİDE ✓
(0.5, 3.0)	İÇERİDE	İÇERİDE ✓
(1.0, 3.9)	İÇERİDE	İÇERİDE ✓
(3.0, 1.0)	İÇERİDE	İÇERİDE ✓
(3.0, 3.0)	DIŞARIDA	DIŞARIDA ✓
(5.0, 1.0)	DIŞARIDA	DIŞARIDA ✓
(-1.0, 2.0)	DIŞARIDA	DIŞARIDA ✓
(2.5, 3.0)	DIŞARIDA	DIŞARIDA ✓

Önemli Not: (3.0, 3.0) noktası L'nin içbükey (boş) köşesine düşmektedir. Algoritma bu noktayı doğru şekilde DIŞARIDA olarak sınıflandırmış; konkav poligon desteğinin çalıştığı kanıtlanmıştır.

4. Hızlanma Katsayıları

Testler Windows 11, Java 8, Intel Core i5 ortamında gerçekleştirilmiştir. Her ölçüm 3 tekrar ile en iyi süre alınmıştır.

4.1 Nokta Sayısına Göre Hızlanma (4 Thread)

Nokta Sayısı	Sıralı (ms)	Paralel 4t (ms)	Hızlanma
100	0.003	0.800	~0× (thread maliyeti)
1.000	0.022	0.735	~0× (thread maliyeti)
10.000	0.299	0.724	0.41×
100.000	3.002	1.715	1.75×
1.000.000	24.16	8.05	3.00×

Yorum: 100 ve 1.000 noktada thread başlatma maliyeti hesaplama süresinden fazla olduğu için paralel sürüm daha yavaş görünür. 100.000 noktadan itibaren paralel sürüm belirgin avantaj kazanmaya başlar. 1.000.000 noktada 4 thread ile 3.00× hızlanma elde edilmiştir.

4.2 Thread Sayısına Göre Hızlanma

Nokta Sayısı	Thread	Sıralı (ms)	Paralel (ms)	Hızlanma
100.000	2	3	2	1.50×
100.000	4	3	1	3.00×
100.000	8	3	1	3.00×
500.000	2	14	7	2.00×
500.000	4	14	4	3.50×
1.000.000	4	24	8	3.00×
1.000.000	8	24	6	4.00×
5.000.000	8	101	30	3.37×

En yüksek hızlanma: 1.000.000 nokta, 8 thread → 4.00× hızlanma. 100.000 noktada 4 thread ile 3.00× hızlanma elde edilmiştir. Küçük veri setlerinde (100–10.000 nokta) thread oluşturma ve yönetim maliyeti hesaplama süresini aştığından paralel sürüm yavaş kalmaktadır. Bu durum Amdahl Yasası ile açıklanır: sabit başlangıç maliyeti, veri büyüdükçe ihmal edilebilir hale gelir.

5. Paralel Çözümün Açıklaması

5.1 Neden Bu Problem Paralleleştirmeye Uygunudur?

Her test noktasının PIP sonucu diğer noktalardan tamamen bağımsızdır. Bu tür problemlere 'embarrassingly parallel' (utanç verici derecede paralel) denir: thread'ler arasında iletişim ya da senkronizasyon gerekmez. Tek paylaşılan yapı sonuç dizisi (boolean[] results) olup her thread farklı indislere yazdığından yarış durumu (race condition) oluşmaz.

5.2 İş Bölümü Stratejisi

M nokta T thread'e $[M/T]$ büyüklüğünde ardışık bloklar hâlinde dağıtılır. Son thread, tam bölünme olmadığında kalan noktaları alır. Her thread `ExecutorService.submit()` ile görevlendirilir ve `Future.get()` çağrısıyla tüm thread'lerin tamamlanması beklenir; ardından executor kapatılır.

5.3 Görselleştirici

`PolygonVisualizer.java` (Swing tabanlı): Sol panel sıralı, sağ panel paralel sonucu eş zamanlı gösterir. Yeşil nokta = poligon içinde, Kırmızı nokta = dışında. Nokta sayısı ve thread sayısı arayüzden değiştirilerek hızlanma farkı canlı izlenebilir.

6. Sonuç

Bu proje, thread tabanlı paralelleştirmenin veri bağımsız problemlerde ne denli etkili olduğunu gerçek ölçüm verileriyle ortaya koymuştur.

- Ray Casting algoritması hem konveks hem konkav poligonlarda doğru çalışmaktadır (8/8 doğruluk testi geçildi).
- 100.000 noktadan itibaren paralel sürüm net avantaj sağlamaktadır.
- 4 thread ile $3.00-3.50\times$, 8 thread ile $4.00\times$ hızlanma elde edilmiştir.
- Küçük veri setlerinde thread oluşturma maliyeti hesaplama süresini geçtiğinden paralel sürüm dezavantajlıdır — bu beklenen ve teorik olarak açıklanabilir bir durumdur.

Pratik öneri: 100.000+ nokta için 4 thread, en iyi maliyet/kazanç oranını sunar.